Telecomm Suport Traige Agent Report

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	ANKIT YADAV	EN22CS301136
2	ANSHUMAN GEETE	EN22CS301160
3	ARTI JAIN	EN22CS301206
4	ATISHA JAIN	EN22CS301235
5	AVANI SHARMA	EN22CS301239
6	ISHANA HATODIYA	EN23CS3L1011

Group Name: DO-19

Project Number: 07D2

Industry Mentor Name: Aashruti Shah

University Mentor Name:

Academic Year: 2022-2026

Problem Statement & Objectives

1. Problem Statement

The project focuses on automating the provisioning of infrastructure for a Telecom system using Infrastructure as Code (IaC) tools such as Terraform and Ansible. Traditional infrastructure setup involves manual configuration, which is time-consuming, error-prone, and difficult to scale.

This project aims to eliminate manual intervention by implementing script-based deployment of cloud infrastructure (AWS) and local Docker environments. It also ensures automated installation of dependencies and deployment of telecom services, enabling a consistent, repeatable, and scalable system setup.

2. Project Objectives

- Automate infrastructure provisioning using Terraform
- Implement configuration management using Ansible
- Enable deployment on both AWS cloud and local Docker environments
- Build a CI/CD pipeline for automated deployment
- Ensure idempotency (same output on repeated runs)
- Reduce human errors in system setup
- Enable scalability of telecom services

3. Scope of the Project

Included:

- Infrastructure provisioning using Terraform
- Configuration management using Ansible

- Docker-based telecom service deployment
- CI/CD automation using GitHub Actions
- AWS deployment (EC2, VPC, Security Groups)

Excluded:

- Telecom business logic implementation
- Deep telecom protocol handling (SIP internals, etc.)
- Production-grade telecom network optimization

Proposed Solution

1. Key features

- Fully automated infrastructure setup
- Multi-environment deployment (Local + Cloud)
- Containerized telecom services using Docker
- CI/CD pipeline for continuous deployment
- Scalable architecture using AWS
- Secure access using IAM roles and secrets
- Monitoring-ready system

2. Overall Architecture / Workflow

The system follows a modular DevOps pipeline:

- Developer pushes code to GitHub
- GitHub Actions CI/CD pipeline is triggered
- Terraform provisions AWS infrastructure (EC2, VPC, Security Groups)
- Ansible configures instances and installs dependencies

- Docker containers are pulled and deployed
- Telecom services (Call Handler, SMS Router, Registry) are started
- Monitoring services are initialized
- Application becomes accessible via public IP

Two Deployment Modes:

- **Cloud Mode:** AWS EC2 deployment
- **Local Mode:** Docker Compose telecom network

3. Tools & Technologies Used

Infrastructure as Code Tool

- **Terraform**
  - Used to provision AWS infrastructure (EC2, VPC, Subnets)
  - Enables version-controlled infrastructure
  - Supports repeatable and automated deployments

Cloud Platform

- **Amazon Web Services (AWS)**
  - EC2 for compute resources
  - VPC for network isolation
  - Security Groups for controlled access
  - S3 (optional) for Terraform state storage

Containerization & Deployment

- **Docker**
  - Used to containerize telecom services
  - Ensures consistency across environments
- **Docker Compose**
  - Used to simulate telecom network locally

Version Control System

- **Git & GitHub**
    - Source code management
    - Collaboration and version tracking
    - Integration with CI/CD pipeline

Backend & Frontend Technologies

- Backend services simulate telecom operations (Call Handler, SMS Router, Registry)
- Monitoring Dashboard built using Streamlit
- APIs used for communication between services

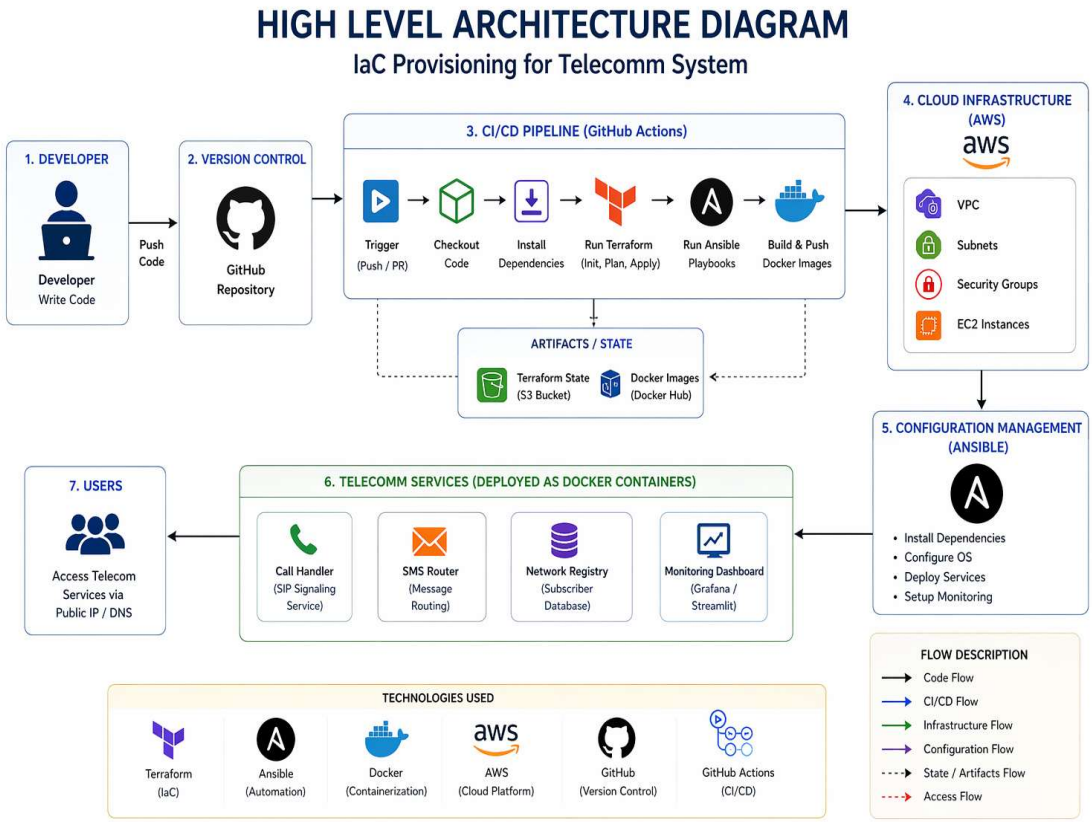
Database

- No traditional database used
- Uses:
    - Registry Service for subscriber data
    - Terraform State Files for infrastructure tracking

CI/CD Pipeline

- **GitHub Actions**
    - Automates build and deployment
    - Executes Terraform and Ansible scripts
    - Deploys Docker containers on AWS

High Level Architecture Diagram

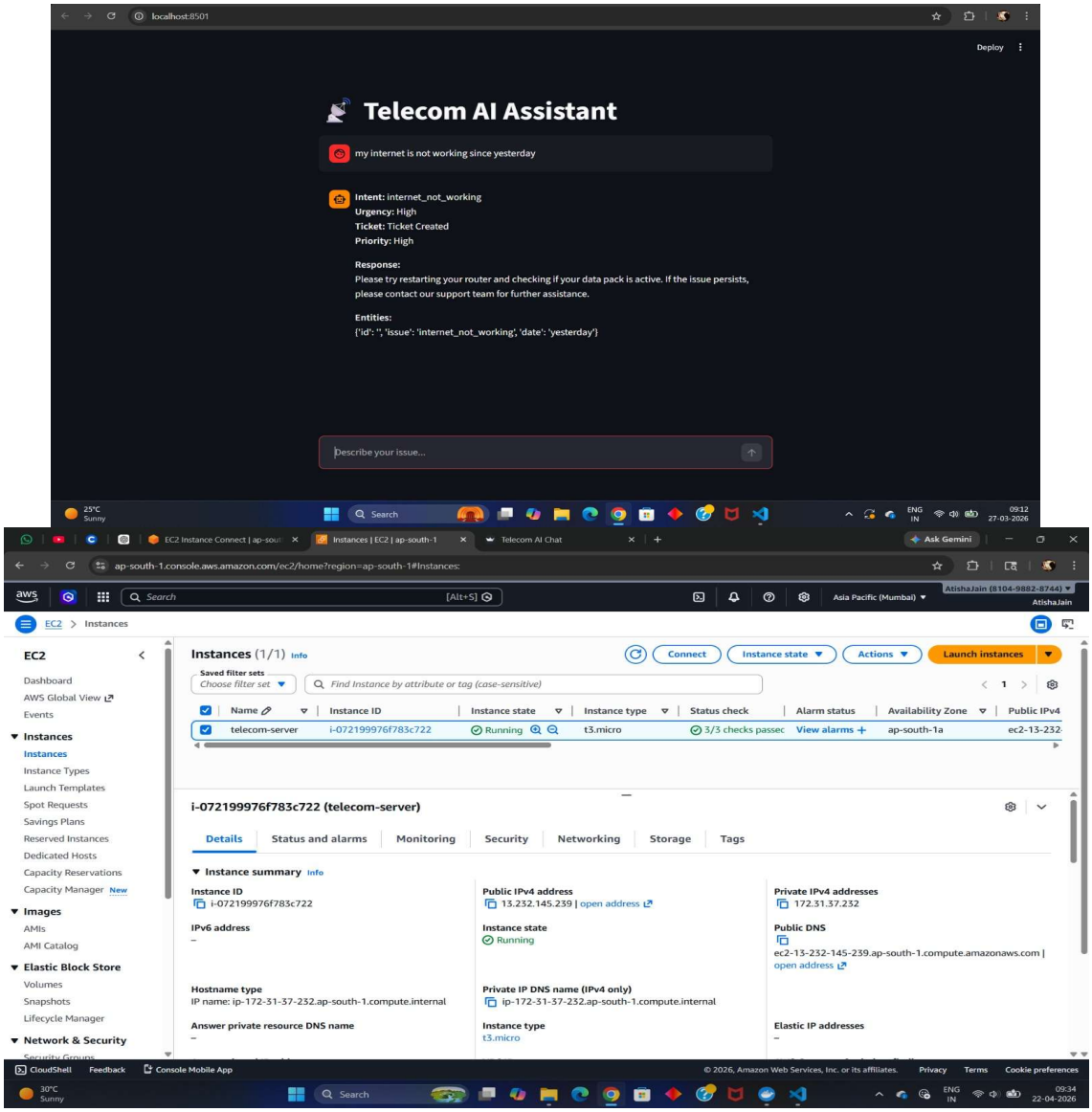


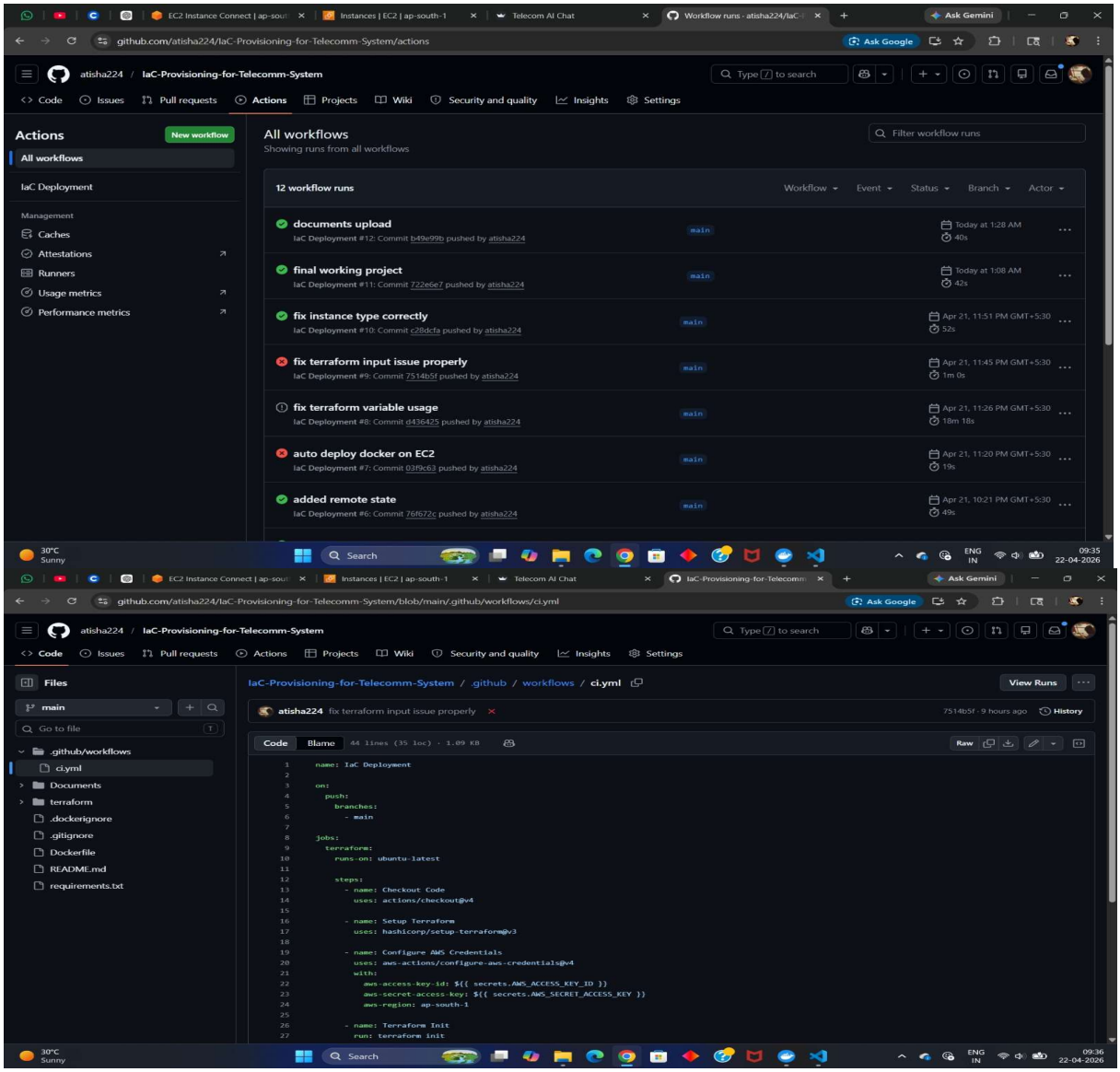
Results & Output

- Successful provisioning of AWS infrastructure using Terraform
- Automated configuration of instances using Ansible
- Deployment of telecom services using Docker containers
- CI/CD pipeline successfully triggered on each code push
- Reduced manual setup time significantly

- Achieved consistent and repeatable deployments

1. Screenshots / outputs




github.com/atisha224/IaC-Provisioning-for-Telecomm-System/actions
atisha224 / IaC-Provisioning-for-Telecomm-System
Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings
Actions
New workflow
All workflows
IaC Deployment
Management
Caches
Attestations
Runners
Usage metrics
Performance metrics
All workflows
Showing runs from all workflows
Filter workflow runs
12 workflow runs
Workflow Event Status Branch Actor
documents upload
IaC Deployment #12: Commit b49e99b pushed by atisha224
main
Today at 1:28 AM
40s
final working project
IaC Deployment #11: Commit 722e6e7 pushed by atisha224
main
Today at 1:08 AM
42s
fix instance type correctly
IaC Deployment #10: Commit c28dcfa pushed by atisha224
main
Apr 21, 11:51 PM GMT+5:30
52s
fix terraform input issue properly
IaC Deployment #9: Commit 7514b5f pushed by atisha224
main
Apr 21, 11:45 PM GMT+5:30
1m 0s
fix terraform variable usage
IaC Deployment #8: Commit d436425 pushed by atisha224
main
Apr 21, 11:26 PM GMT+5:30
18m 18s
auto deploy docker on EC2
IaC Deployment #7: Commit 03f9c63 pushed by atisha224
main
Apr 21, 11:20 PM GMT+5:30
19s
added remote state
IaC Deployment #6: Commit 76f672c pushed by atisha224
main
Apr 21, 10:21 PM GMT+5:30
49s
github.com/atisha224/IaC-Provisioning-for-Telecomm-System/blob/main/.github/workflows/ci.yml
Files
main
Go to file
.github/workflows
ci.yml
Documents
terraform
.dockerignore
.gitignore
Dockerfile
README.md
requirements.txt
IaC-Provisioning-for-Telecomm-System / .github / workflows / ci.yml
View Runs
atisha224 fix terraform input issue properly
7514b5f · 9 hours ago
History
Code Blame 44 lines (35 loc) · 1.09 KB
Raw
name: IaC Deployment
on:
push:
branches:
- main
jobs:
terraform:
runs-on: ubuntu-latest
steps:
- name: Checkout Code
uses: actions/checkout@v4
- name: Setup Terraform
uses: hashicorp/setup-terraform@v3
- name: Configure AWS Credentials
uses: aws-actions/configure-aws-credentials@v4
with:
aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
aws-region: ap-south-1
- name: Terraform Init
run: terraform init

2. Key outcomes

- Achieved fully automated infrastructure provisioning
- Reduced deployment time and manual errors
- Built a scalable and reusable DevOps pipeline
- Successfully integrated multiple DevOps tools
- Demonstrated real-world IaC implementation

Conclusion

The project successfully demonstrates the use of Infrastructure as Code (IaC) in automating the provisioning and deployment of a telecom system. By integrating Terraform, Ansible, Docker, and AWS, the system achieves a high level of automation, scalability, and reliability.

The CI/CD pipeline further enhances efficiency by enabling continuous deployment with minimal manual intervention. Overall, the project reflects industry-standard DevOps practices and provides a strong foundation for real-world infrastructure automation

Future Scope & Enhancements

- Implement Kubernetes (K8s) for container orchestration
- Add Auto Scaling Groups in AWS
- Enhance monitoring with advanced alerting systems
- Add Load Balancer (ELB) for better traffic management
- Integrate security scanning tools in CI/CD pipeline
- Extend telecom simulation to real-world protocols